

# Class 08: React Hooks

## 2110507 Software Development Practice II

Machigar Ongtang, Ph.D.

# Hooks



- React มาพร้อมชุดของ function ที่เรียกว่า Hooks ที่ใช้ในการเพิ่ม logic การทำงานเพิ่มเติมเข้าไปให้กับ component
  - เป็น function ที่พร้อมใช้งานเฉพาะเมื่อ React กำลัง render และใช้เพื่อเกี่ยว (hook) เข้ากับ React Features ต่าง ๆ
  - ใช้ในการควบคุม State และ Lifecycle ของ functional components ต่าง ๆ
- Function เหล่านี้ขึ้นต้นด้วยคำว่า `use`
  - React มาพร้อม built-in Hooks เช่น `useState` , `userReducer`, `useEffect` เป็นต้น
  - Developer สามารถ customize Hooks ได้ โดย combine hook ที่มีอยู่
- Hooks เป็นฟังก์ชั่นที่มีเงื่อนไขเข้มงวดกว่า functions ทั่วไป
  - เรียก Hooks ได้จาก top level ของ component (หรือ Hooks อื่น) เท่านั้น
    - ไม่ควรเรียก hook ได้ภายในเงื่อนไข หรือใน loop หรือ nested function เนื่องจากลำดับของการ execute hook อาจผิดไป เมื่อ component ถูก render
    - แม้ว่า hook เป็น function แต่ทำงานคล้าย unconditional declaration ที่ประกาศถึง feature ที่ component นั้น ๆ ต้องการ

# useRef



- useRef Hook ช่วยให้คงค่า value ข้ามการ render ใช้เก็บ mutable value โดย ไม่ทำให้เกิดการ re-render เมื่อมีการ update และใช้ในการเข้าถึง DOM element โดยตรง
  - ใช้งานโดยการ import

```
import { useRef } from 'react'
```

  - ประกาศ ref ไว้ใน top-level ของ component โดยทำงานบน client component (‘use client’)

```
const refName = useRef(initialValue)
```

    - *initialValue* ค่าเริ่มต้นของ reference object โดยเป็นข้อมูลชนิดใดก็ได้
    - *refName* ชื่อ reference object

  - ตัวอย่าง

```
const inputRef = useRef(null)
const intervalRef = useRef(0)
```

# useRef

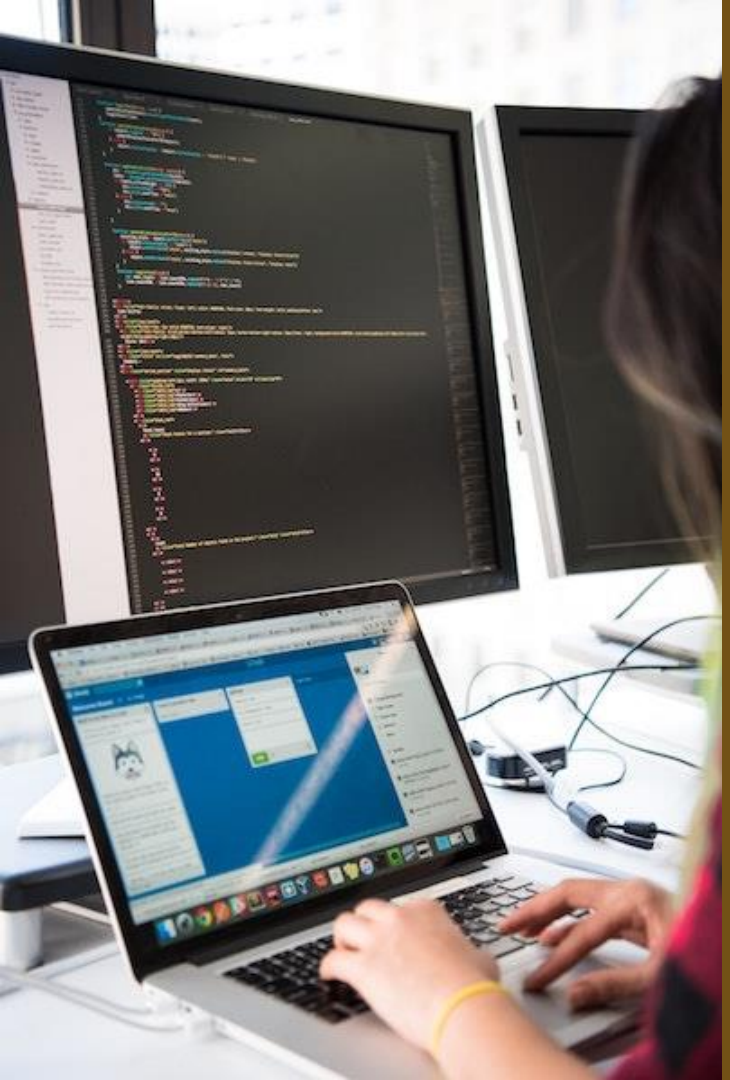


- การใช้งาน Reference Object
  - เข้าถึง หรือตั้งค่าปัจจุบันของ reference object ผ่าน property `refName.current`
  - ตัวอย่าง `intervalRef.current = 10`
  - การอับเดตค่า `refName.current` ไม่ทำให้เกิดการ re-render ใหม่ เนื่องจากเป็น plain JavaScript object ที่ React ไม่ได้ตรวจจับการเปลี่ยนแปลง
  - ไม่ควรอ่านหรือเขียนค่า `refName.current` ในระหว่างการ render ยกเว้นการ initialization เนื่องจากจะให้พฤติกรรมที่คาดเดาไม่ได้

- การเข้าถึง DOM Element ผ่าน Ref Object
  - React JSX ที่นำเสนอ DOM node รองรับ `ref` attribute
  - กำหนด Ref Object เป็นค่าของ `ref` attribute ดังตัวอย่าง

```
<input ref={inputRef} />
```

    - หลังจากที่ React สร้าง DOM node แล้ว จะตั้งค่า `inputRef.current` ให้เป็น DOM node นี้ ทำให้อ้างมายัง DOM node นี้ผ่าน `inputRef` ได้



# Workshop 8.1:
# useRef vs. Local Variable

# useRef vs. Local Variable



เปรียบเทียบการเก็บข้อมูลใน local variable กับผ่าน useRef ใน components/CarPanel.tsx

- ลองสร้างปุ่ม พร้อมการนับจำนวนครั้งของการ click ลงในหน้า CarPanel.tsx
- เก็บข้อมูลใน local variable
  - สร้าง local variable count

```
export default function CarPanel() {

    let count = 0

    const compareReducer = ( compareList:Set
```

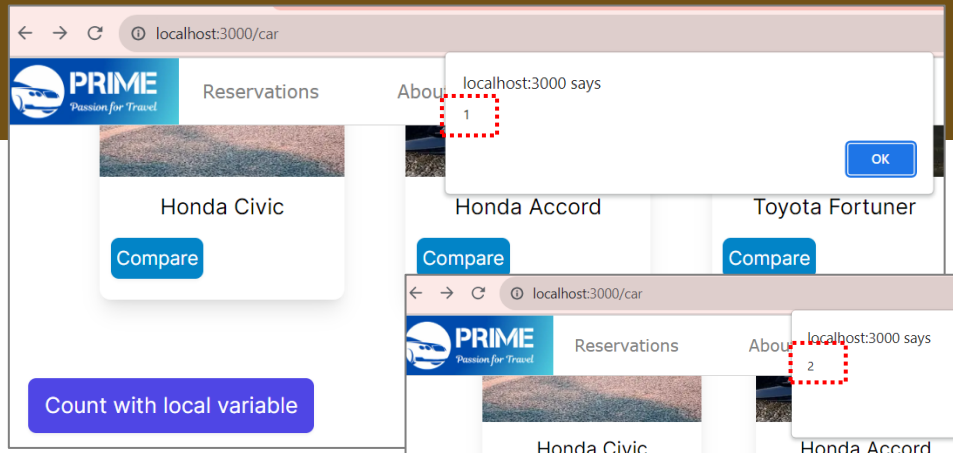
  - สร้างปุ่มให้เมื่อกด แล้วนับจำนวนการคลิก พร้อม alert ออกมา

```
<div className="w-full text-xl font-medium">Compare List: { compareList.size
{ Array.from(compareList).map( (car)=><div key={car}
    onClick={(e)=>{e.stopPropagation(); dispatchCompare({type:'remove', carNa
    {car}</div> ) }

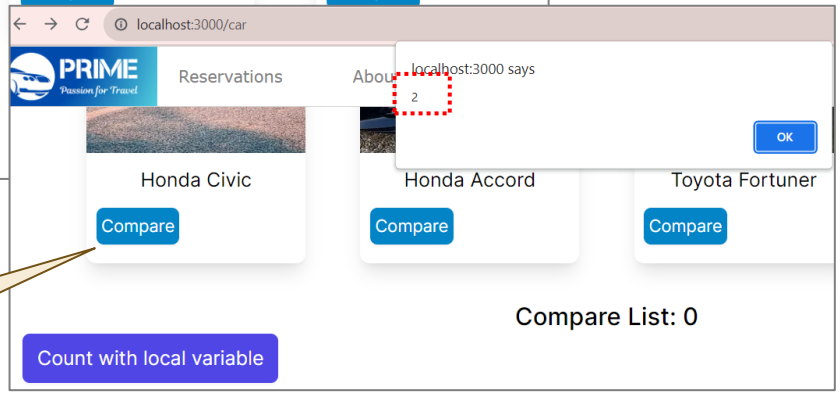
<button className="block rounded-md bg-sky-600 hover:bg-indigo-600 px-3 py-2
text-white shadow-sm"
onClick={()=> {count = count+1; alert(count)} }>
    Count with local variable
</button>
```

# useRef vs. Local

- เก็บข้อมูลใน local variable
  - ทดสอบการทำงาน

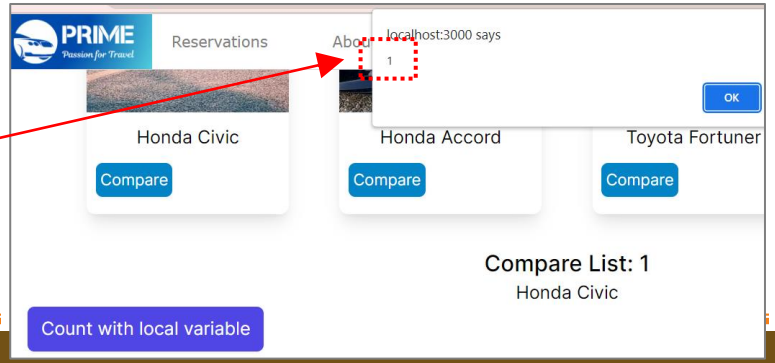



- หลังจาก Click ปุ่มไปหลายรอบ ลองกดปุ่ม Compare ซึ่งเป็นการเปลี่ยน state variable และทำให้มีการ re-render หน้านี้




กดปุ่ม compare เป็น การเปลี่ยน state variable และทำให้มีการ re-render หน้านี้

- หลังจากนั้น เมื่อกด Click ปุ่ม จะพบว่าจำนวนคลิกกลับมาเป็น 1 อีกรอบ เพราะการ re-render ทำให้ local variable ถูกรีเซ็ตค่า



# useRef vs. Local Variable

- เก็บข้อมูลใน Ref
  - เปลี่ยน local variable count เป็น Ref Object countRef

```
1   'use client'
2   import { useReducer } from "react"
3   import ProductCard from "./ProductCard"
4   import Link from "next/link"
5   import { useRef } from "react"
6
7   export default function CarPanel() {
8
9       const countRef = useRef(0)
```

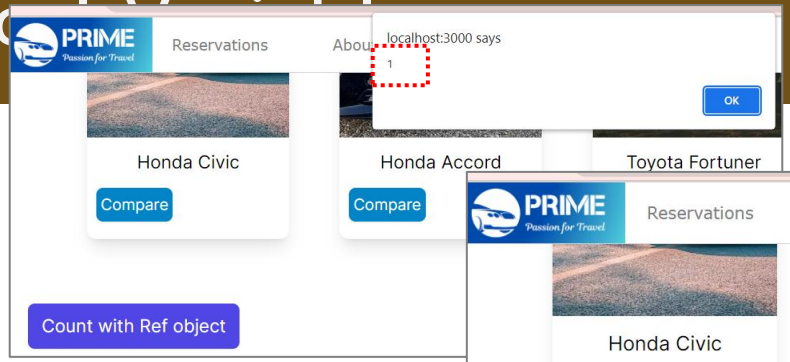
  - เปลี่ยนปุ่มให้ เมื่อกด แล้วนับจำนวนการคลิกด้วย countRef พร้อม alert ออกมา

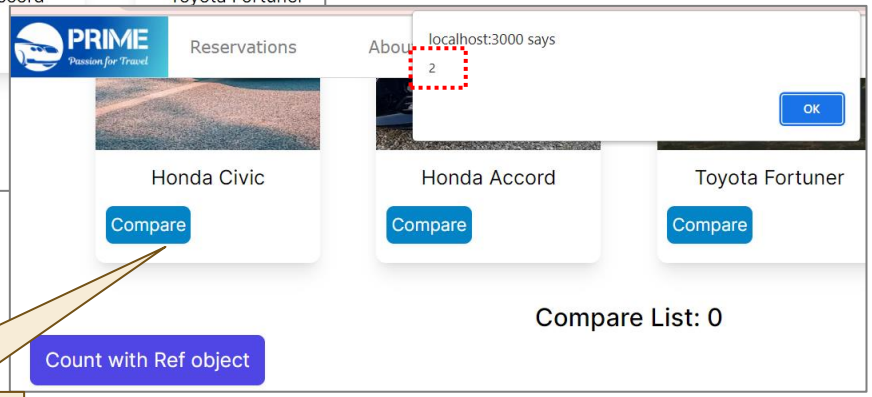
```
<button className="block rounded-md bg-sky-600 hover:bg-indigo-600 px-3 py-2
text-white shadow-sm"
onClick={()=> {countRef.current = countRef.current+1; alert(countRef.current)} }>
    Count with Ref object
</button>
```

# useRef vs. Local Variable



- เก็บข้อมูลใน Ref object
  - ทดสอบการทำงาน

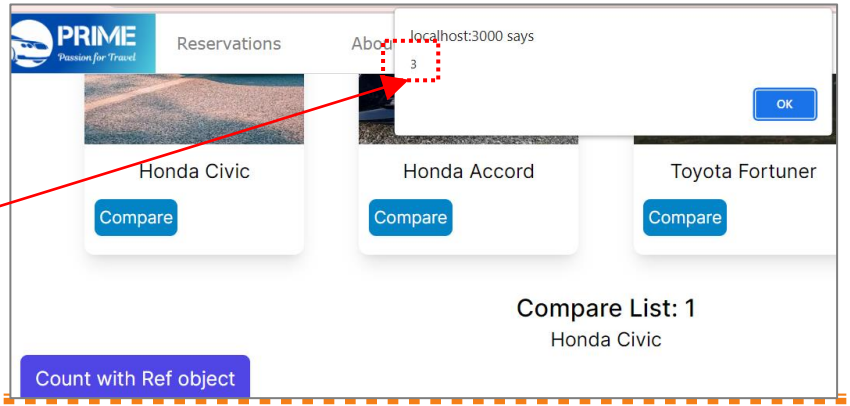


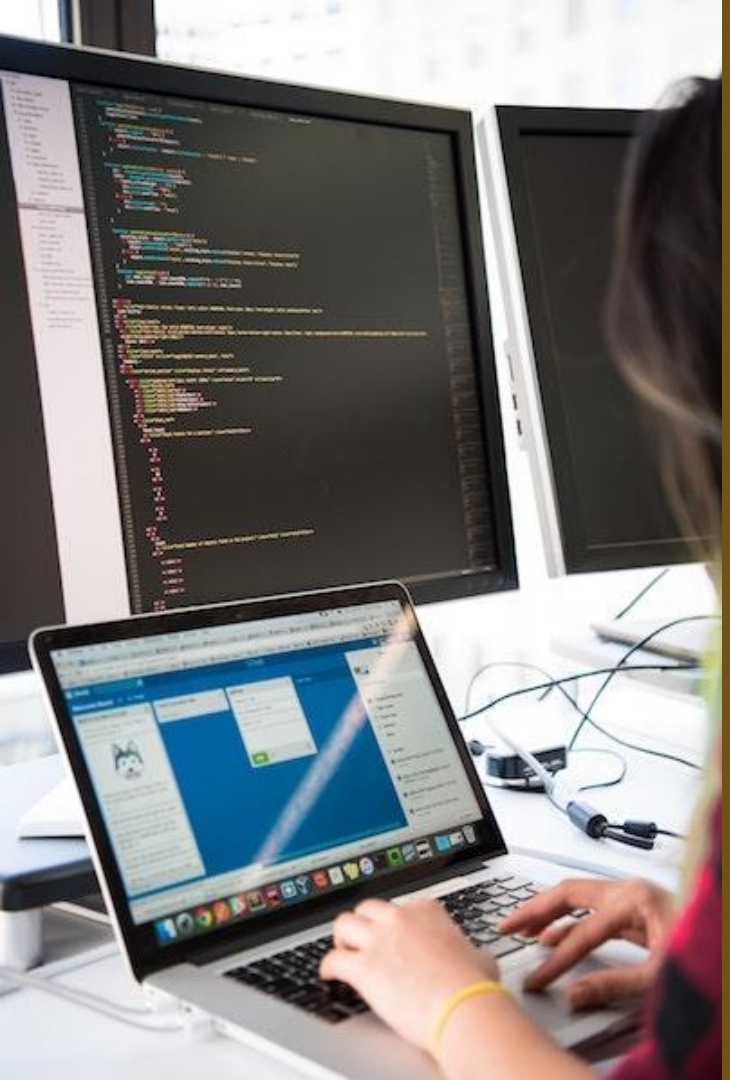




- หลังจาก Click ปุ่มไปหลายรอบ ลองกดปุ่ม Compare ซึ่งเป็นการเปลี่ยน state variable และทำให้มีการ re-render หน้านี้

กดปุ่ม compare เป็น การเปลี่ยน state variable และทำให้มีการ re-render หน้านี้

- หลังจากนั้น เมื่อกด Click ปุ่ม จะพบว่าจำนวนคลิกนับต่อจากเดิม เพราะRef object รักษาค่า ข้ามการ render





# Workshop 8.2:
# Access DOM with useRef

# Access DOM with useRef



- ลองการดึง Focus มาที่กล่อง Text Input เมื่อกดปุ่ม
  - ใน CarPanel สร้าง Ref Object ไปยัง HTML Input Element

```
export default function CarPanel() {

    const countRef = useRef(0)
    const inputRef = useRef<HTMLInputElement>(null)
```

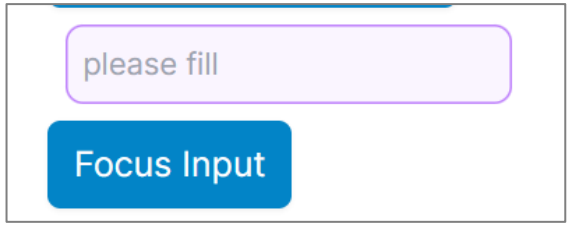
  - สร้าง Text Input พร้อม Ref Object มายัง Text Input นี้

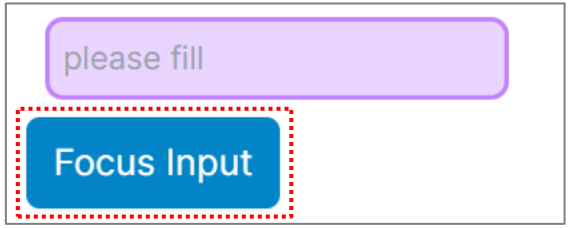
```
onClick={()=> {countRef.current = countRef.current+1; alert(countRef.current)} }>
    Count with Ref object
</button>
<input type="text" placeholder="please fill" className="block text-gray-900 text-sm
rounded-lg p-2 m-2 bg-purple-50 ring-1 ring-inset ring-purple-400
focus:outline-none focus:bg-purple-200 focus:ring-2"
ref={inputRef}/>
```

# Access DOM with useRef

- ลองการดึง Focus มาที่กล่อง Text Input เมื่อกดปุ่ม
  - สร้างปุ่ม และเมื่อกดปุ่ม ให้ focus ไปที่ Text Input ที่ Ref Object อ้างอิงอยู่

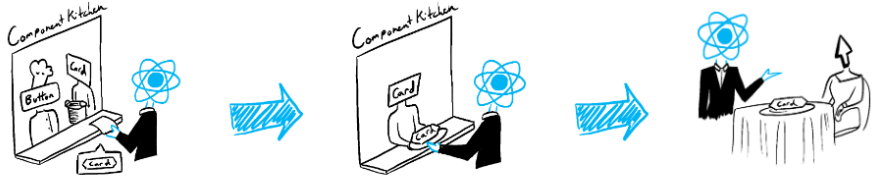
```
<input type="text" placeholder="please fill" className="block text-gray-900 te
rounded-lg p-2 m-2 bg-purple-50 ring-1 ring-inset ring-purple-400
focus:outline-none focus:bg-purple-200 focus:ring-2"
ref={inputRef}/>
<button className="block rounded-md bg-sky-600 hover:bg-indigo-600 px-3 py-2
text-white shadow-sm"
onClick={()=> {if(inputRef.current!=null) inputRef.current.focus()} }>
    Focus Input
</button>
```



# Hook for React Effects

- Logic พื้นฐานใน React Component
  - **Rendering** : กระทำโดย code ที่อยู่ระดับ top level ของ component รับหรือนำ props และ state มา transform แล้ว return ออกไปเป็น JSX ที่นำเสนอสิ่งที่ต้องการให้แสดงบนหน้าจอ
    - Rendering code เป็นการประมวลผลหรือการคำนวณที่ตรงไปตรงมา เช่น อาจมี math formula เพื่อการคำนวณผลลัพธ์ เป็นต้น
    - การ render ประกอบด้วย 3 ขั้นตอน
      (1) Trigger การ render
      (2) การทำ rendering
      (3) การ commit ไปยัง DOM




*https://react.dev/learn/render-and-commit*

  - **Event handlers** : เป็น nested function ภายใน component ที่กระทำการต่าง ๆ ตาม event และกระทำ action ที่ซับซ้อนอื่น ๆ
    - เช่น การ update input field, การส่ง HTTP POST request ไปยัง server, การ navigate ไปยังหน้าจออื่น
    - บรรจุ side effect เช่น เปลี่ยน state โดยเกิดจากการกระทำของ user เช่น เมื่อ click ปุ่ม หรือเมื่อพิมพ์ข้อความ

# Hook for React Effects



- **Logic พื้นฐาน 2 รูปแบบ ไม่เพียงพอสำหรับบางสถานการณ์**

  ต้องการ connect ไปยัง server เพื่อรับส่งข้อมูล เมื่อ component แสดงบนหน้าจอ

  - ไม่เป็นการประมวลผลหรือการคำนวณที่ตรงไปตรงมา แต่เป็น side effect รูปแบบหนึ่ง ไม่ใช่ rendering
  - ไม่ได้ตอบสนองต่อ event หนึ่ง event ใดเดี่ยว ๆ ไม่ใช่ Event handlers

- **Effects ใน React สนับสนุนการระบุ side effect ที่เกิดจากการ rendering ไม่ได้เกิดจาก event ใด event หนึ่ง**

  - Effect เกิด แม้ว่าจะไม่มี interaction กับ component เช่น เกิดการเชื่อมต่อไปยัง server
  - Effect จะ run เมื่อสิ้นสุดขั้นตอนการ commit ของการ render หลังการทำ screen update
  - รองรับการ run code หลังการ render เพื่อ **synchronize component กับระบบอื่นที่อยู่นอก React** เช่น การทำงานที่เกี่ยวข้องกับ network หรือ third-party library
  - ใช้ Effect เมื่อต้องการ**การทำงานนอก React code และ synchronize กับ external system**
    - เช่น ทำงานกับ browser APIs, third-party widgets, network ฯลฯ

# useEffect Hook



- การสร้าง Effect ประกอบด้วยขั้นตอนต่อไปนี้
  - Declare การใช้งาน Effect โดย default จะ run ภายหลังการ render
  - ระบุ Effect Dependencies เพื่อกำหนดเงื่อนไขการควบคุมการ run effect เพราะปกติ effect ไม่ควร run ทุกครั้งที่มีการ render แต่ re-run ตามความจำเป็น
    - Connect ไปยัง server เมื่อแสดง component บนหน้าจอ หรือเมื่อมีการเปลี่ยนแปลงบางอย่าง
    - Run animation เมื่อแสดง component
  - Clean-up ระบุเงื่อนไขการ หยุด /undo/clean effect
    - Disconnect หลังจากการ connect
    - Unsubscribe หลังการ subscribe
    - Cancel หรือ ignore หลังการ run การทำงานบางอย่าง

- useEffect เป็น React Hook สำหรับใช้ Effect เพื่อ synchronize component กับระบบภายนอก
  - ใช้โดยการ import

```
import { useEffect } from 'react'
```

# useEffect Hook

- ใช้งาน `useEffect` ที่ top-level ของ component
  - ทำงานบน client component (`'use client'`)
  - ตัวอย่างการใช้งานอย่างง่าย

```
function TestComponent() {
  useEffect(() => {
      ใน Setup Function : Code ส่วนนี้จะทำงานทุกการ render โดยทำเมื่อสิ้นสุดการ commit ของการ render หลังการทำ screen update
  })
    return(
        JSX component
    )
}
```

ใช้งาน `useEffect` อย่างง่าย
- ระบุเพียงส่วนการ Declare การใช้งาน Effect
- ไม่มีการระบุ Effect Dependencies จะทำให้ effect ทำงานในทุกการ render
- ไม่มีการระบุการ Clean-up ทำให้ไม่มีการ clean up ก่อนการ run effect ซ้ำ

# useEffect Hook



- ใช้งาน `useEffect` โดยระบุเงื่อนไขอื่น ๆ
  - `dependencyArray` ระบุเงื่อนไขเพื่อควบคุมการ re-run Effect
    - ไม่ระบุ : Run ในทุกการ render
    - [] : Empty Array ให้ Run ในการ render ครั้งแรก (เมื่อ mount component)
    - [*var1*, *var2*] : ให้ Run ในการ render ครั้งแรก และเมื่อค่า *var1* หรือ *var2* เปลี่ยน

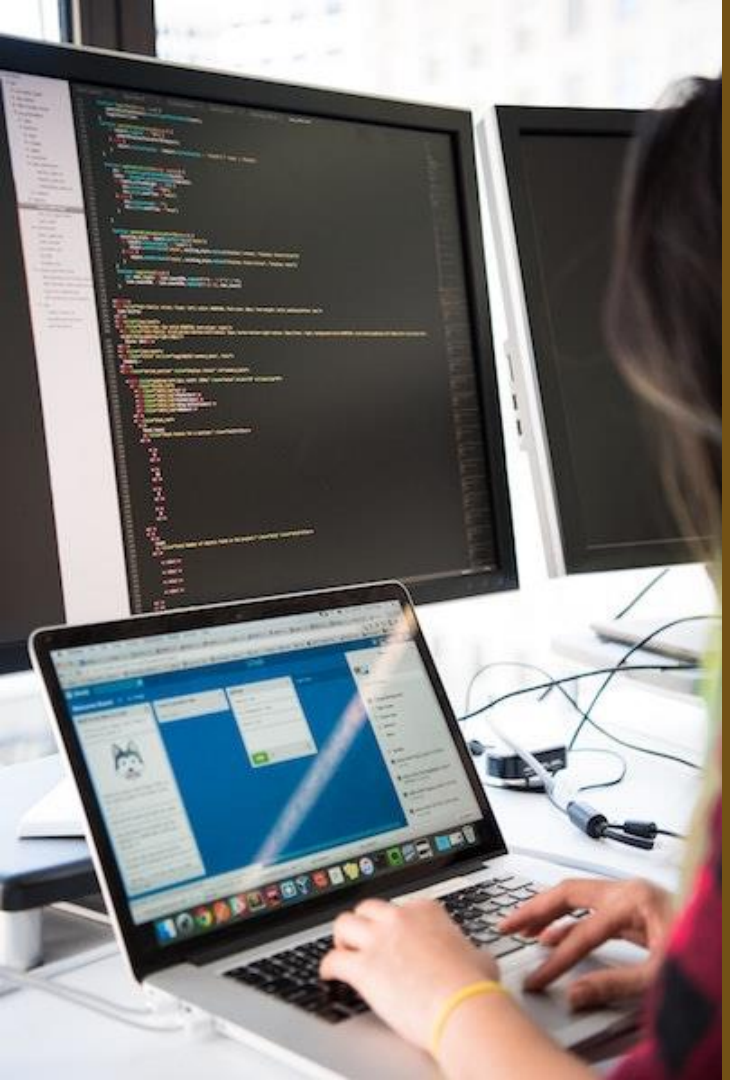
```
useEffect(() => {

      return cleanupFunction

}, dependencyArray )
```

*ใน Setup Function : Code ส่วนนี้จะทำงานเพื่อสร้าง effect*

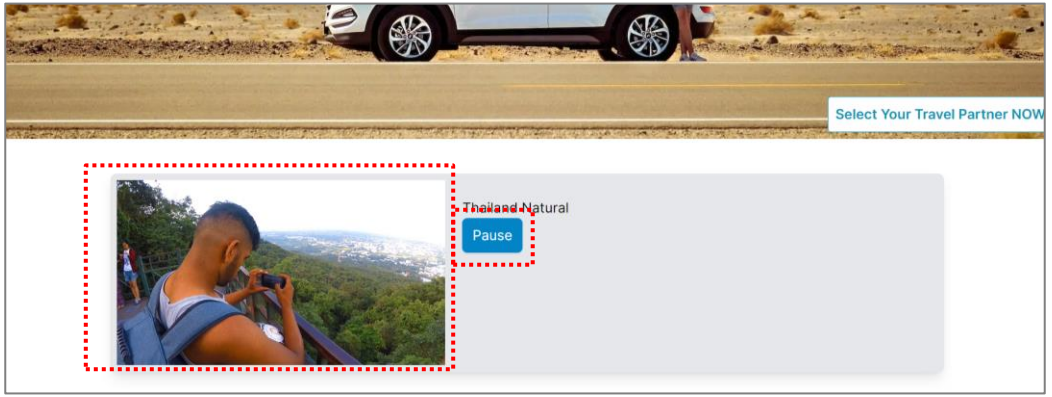
  - `cleanupFunction` ระบุการทำงานเพื่อ หยุด /undo/clean effect
    - เช่น มีการเปิด modal ไว้ ก็ทำการปิด modal (มิเช่นนั้น หาก effect ทำงานอีกรอบ จะพยายามเปิดซ้ำ)
    - Subscribe ใช้งาน API ไว้ ก็ทำการ unsubscribe (มิเช่นนั้น หาก effect ทำงานอีกรอบ จะพยายาม subscribe ซ้ำ)
    - Connect ไปยัง Server ก็ทำการ disconnect (มิเช่นนั้น หาก effect ทำงานอีกรอบ จะพยายาม connect เพิ่ม)



# Workshop 8.3: useEffect

# Controlling VDO with useEffect

- สร้าง Travel VDO Card ไว้บนหน้าแรกของ Car Rental App โดยใช้ `useEffect` เพื่อเข้าถึง control video element




- สร้าง `components\VlogPlayer.tsx` เพื่อบรรจุ VDO โดยรับการควบคุม play / pause จาก component อื่น

```tsx
'use client'
import { useRef } from 'react'
export function VlogPlayer(
    {vdoSrc, isPlaying}: {vdoSrc:string, isPlaying:boolean} ) {

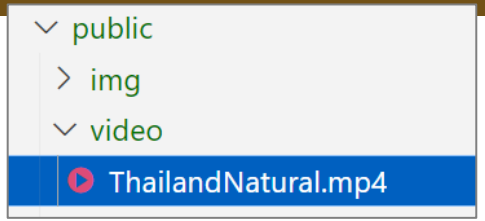
        const vdoRef = useRef<HTMLVideoElement>(null)

    return (
        <video className='w-[40%]' src={vdoSrc} ref={vdoRef} muted
        loop controls/>
    )
}
```

# Controlling VDO with useEffect



- สร้าง `components\TravelCard.tsx` เป็น Card ฝัง `VlogPlayer` และข้อความบรรยาย
  - สร้างโฟลเดอร์ `video` ในโฟลเดอร์ `public` วางไฟล์ video ไว้




```
'use client'
import { VlogPlayer } from "./VlogPlayer"
import { useState } from "react"
export function TravelCard () {

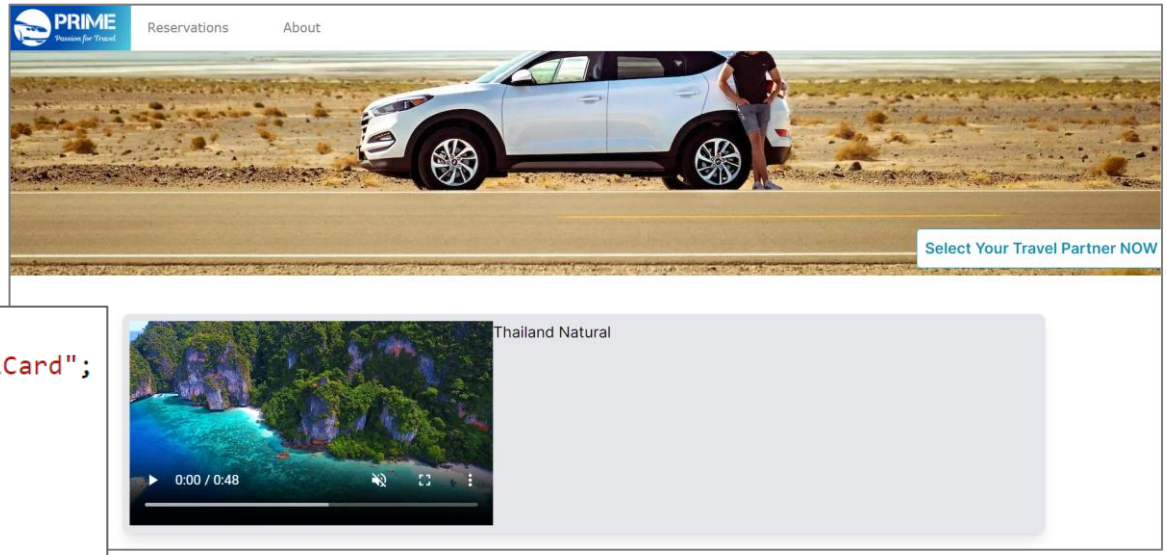
    const [playing, setPlaying] = useState(true)
    return (
        <div className='w-[80%] shadow-lg mx-[10%] my-10 p-2 rounded-lg bg-gray-200
        flex flex-row'>
        <VlogPlayer isPlaying={playing} vdoSrc="/video/ThailandNatural.mp4"></VlogPlayer>
        <div>
            Thailand Natural
        </div>
        </div>
    )
}
```

# Controlling VDO with useEffect

- วาง `TravelCard` เป็น Card ไว้บนหน้าแรกของเว็บ `app\page.tsx`

```tsx
import Banner from "@/components/Banner";
import {TravelCard} from "@/components/TravelCard";

export default function Home() {
  return (
    <main>
      <Banner/>
      <TravelCard></TravelCard>
    </main>
  );
}
```



# Controlling VDO with useEffect



- ใน `components\VlogPlayer.tsx` ลองเข้าถึง video DOM element จาก top level ของ component โดยไม่ใช้ `useEffect`

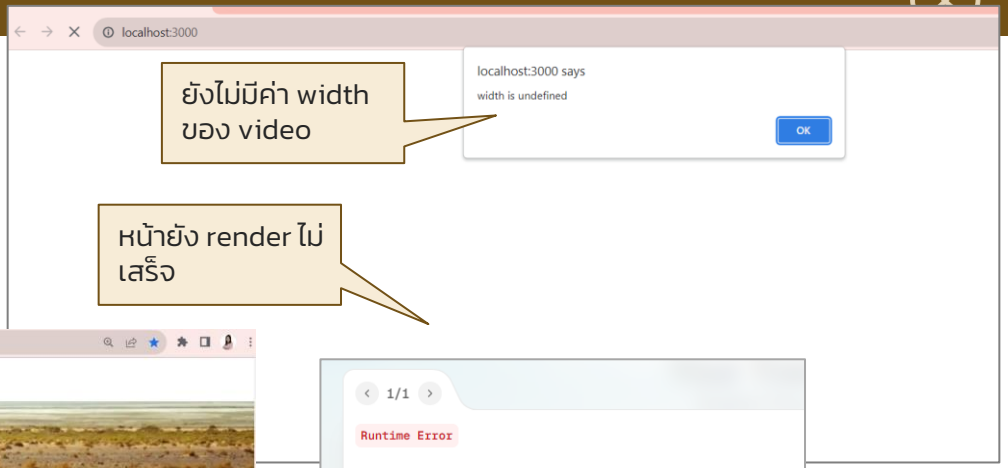
```
'use client'
import { useRef } from 'react'
export function VlogPlayer(
    {vdoSrc, isPlaying}: {vdoSrc:string, isPlaying:boolean} ) {

    const vdoRef = useRef<HTMLVideoElement>(null)
    alert('width is ' + vdoRef.current?.videoWidth)
    if(isPlaying) {
        vdoRef.current?.play()
    }
    else {
        vdoRef.current?.pause()
    }

    return (
        <video className='w-[40%]' src={vdoSrc} ref={vdoRef} muted
```

# Controlling VDO with useEffect

- ลอง Refresh ดูจะพบว่า code เหล่านั้น run ตั้งแต่ DOM ยัง render ไม่เสร็จ จึงไม่สามารถเข้าถึง Video DOM element เพื่อใช้งานได้










ยังไม่มี video ให้ ref object ขณะสั่ง play จึงไม่ play




Note: สำหรับ Next.js 15 การใช้ alert ซึ่งเข้าถึง Browser function ตั้งแต่ขณะที่ code ยัง run บน server จะมี error ขึ้นดังรูป

# Controlling VDO with useEffect



- ใน components\VlogPlayer.tsx ลองเข้าถึง video DOM element จาก top level ของ component ด้วย useEffect

```tsx
import { useEffect } from 'react'
export function VlogPlayer(
    {vdoSrc, isPlaying}: {vdoSrc:string, isPlaying:boolean} ) {

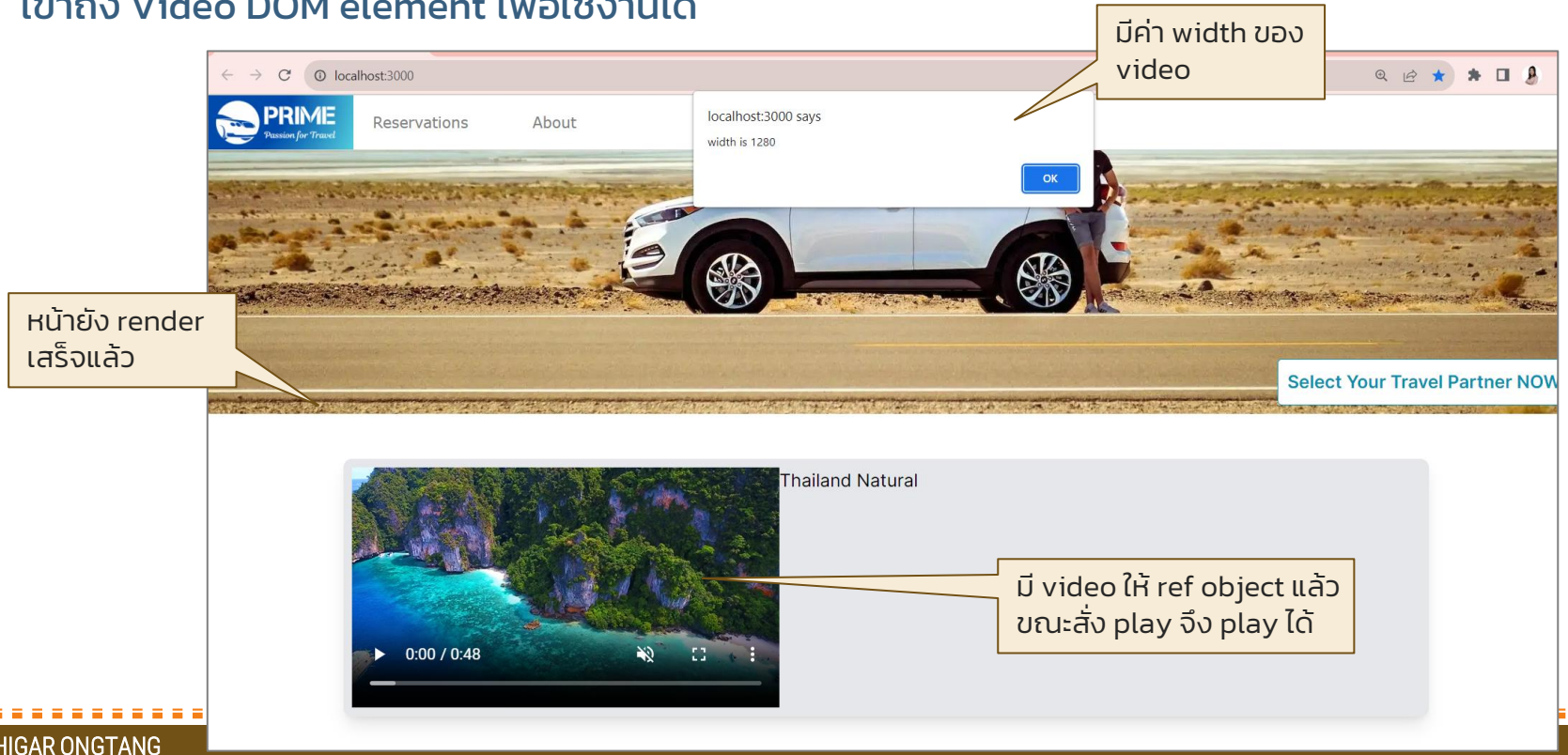
    const vdoRef = useRef<HTMLVideoElement>(null)

    useEffect(()=> {
        alert('width is ' + vdoRef.current?.videoWidth)
        if(isPlaying) {
            vdoRef.current?.play()
        }
        else {
            vdoRef.current?.pause()
        }
    })

    return (
```

# Controlling VDO with useEffect

- ลอง Refresh ดูจะพบว่า code เหล่านั้น run หลังจากที่ DOM รับการ render เสร็จแล้ว จึงสามารถเข้าถึง Video DOM element เพื่อใช้งานได้



# Controlling VDO with useEffect



- ใน components\VlogPlayer.tsx ลบการ alert เพื่อไม่ให้ alert ทุกการ render

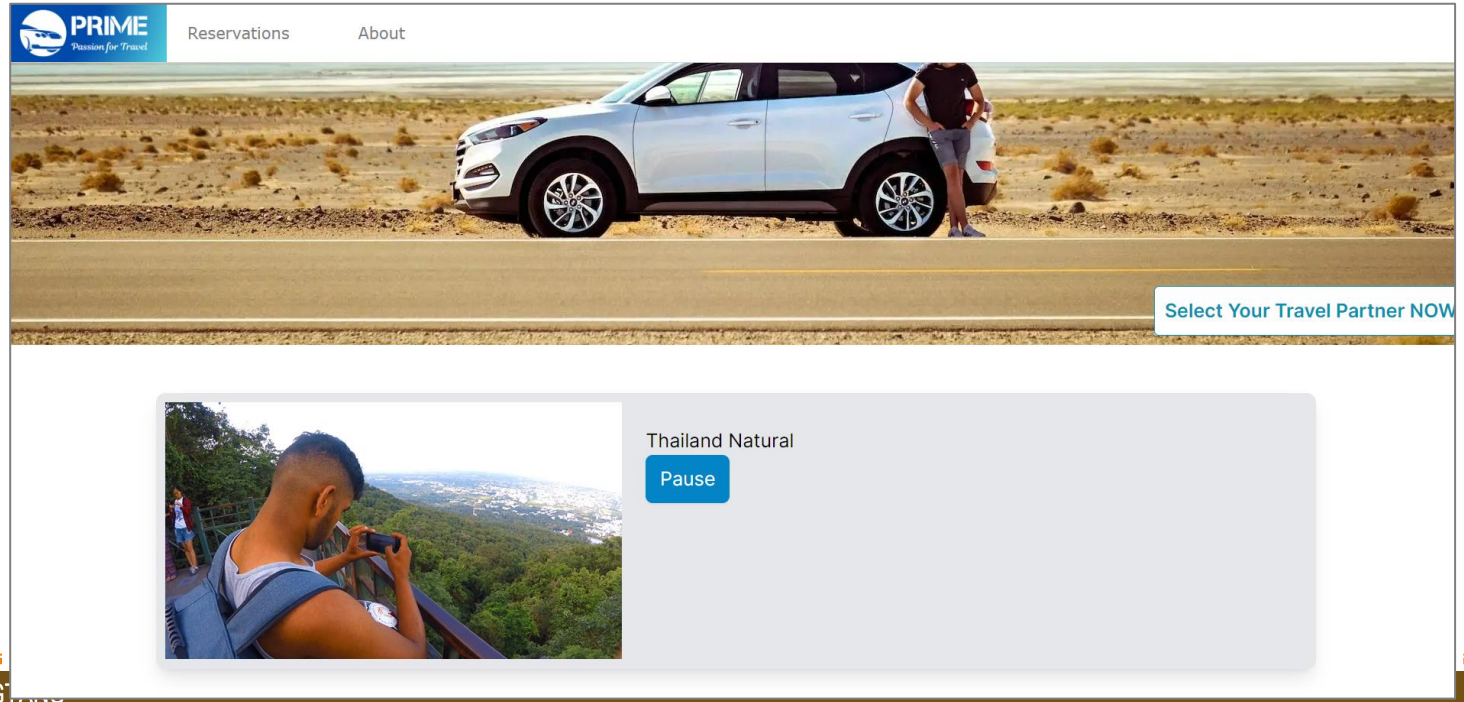
```
useEffect(()=> {
    //alert('width is ' + vdoRef.current?.videoWidth)
    if(isPlaying) {
        vdoRef.current?.play()
```

- ใน TravelCard.tsx เพิ่มปุ่มควบคุมการ play/pause ผ่าน state ใน TravelCard.tsx

```
<VlogPlayer isPlaying={playing} vdoSrc="/video/ThailandNatural.mp4"></VlogPlayer>
<div className="m-5">
    Thailand Natural
    <button className="block rounded-md bg-sky-600 hover:bg-indigo-600 px-3 py-2
    text-white shadow-sm"
    onClick={()=> setPlaying(!playing) }>
        { playing? 'Pause':'Play' }
    </button>
</div>
```

# Controlling VDO with useEffect

- ทดลองการทำงาน
  - การกด Play/Pause เป็นการเปลี่ยน state ใน `TravelCard` และทำให้ `TravelCard` re-render และ `VlogPlayer` ถูก re-render ด้วย



# Dependencies in useEffect



- การไม่มี dependency array ทำให้ `useEffect` ทำงานในทุกการ render ซึ่งเป็นการทำงานที่ไม่จำเป็น
- ใน `TravelCard` เพิ่ม `Rating` element เข้าไป
  - มีการใช้งาน state `Rating` เพื่อเก็บค่า Rating
  - เปลี่ยนค่าใน state `Rating` ตามค่า Rating ที่ผู้ใช้กำหนด

```
'use client'
import { VlogPlayer } from "./VlogPlayer"
import { useState } from "react"
import { Rating } from '@mui/material';

export function TravelCard () {

    const [playing, setPlaying] = useState(true)
    const [rating, setRating] = useState(0)
```

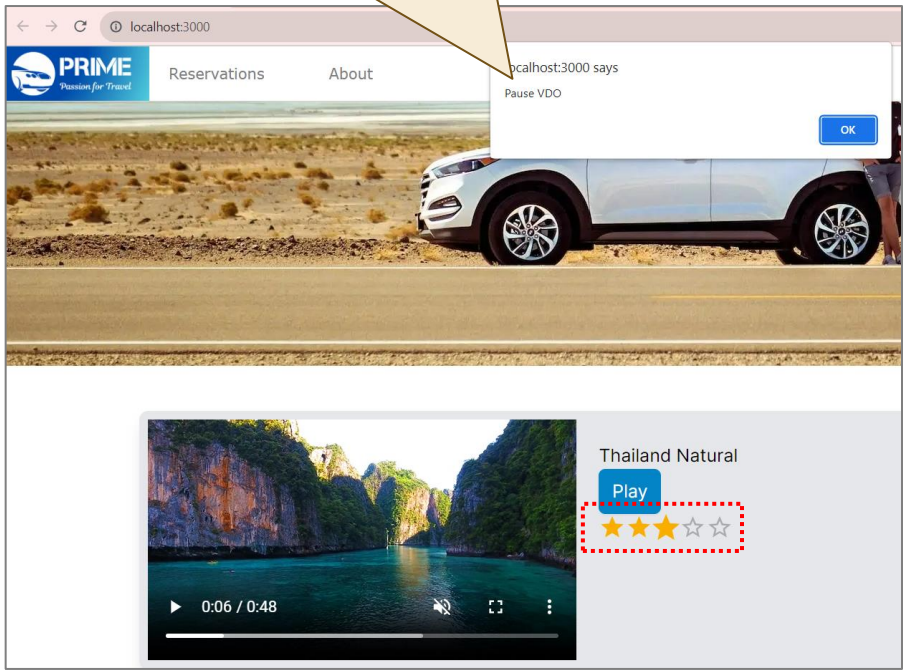
```
    </button>
    <Rating className='w-full h-[10%]' value={(rating==undefined)? 0:rating}
    onChange={(e, newValue) => {if(newValue!=null) setRating(newValue)}}/>
</div>
```

# Dependencies in useEffect

- ใน `components\VlogPlayer.tsx`
  ใส่ alert เพื่อดูการทำงานของ useEffect

```
useEffect(()=> {
    //alert('width is ' + vdoRef.current
    if(isPlaying) {
      alert('Play VDO')
      vdoRef.current?.play()
    }
    else {
      alert('Pause VDO')
      vdoRef.current?.pause()
    }
})
```

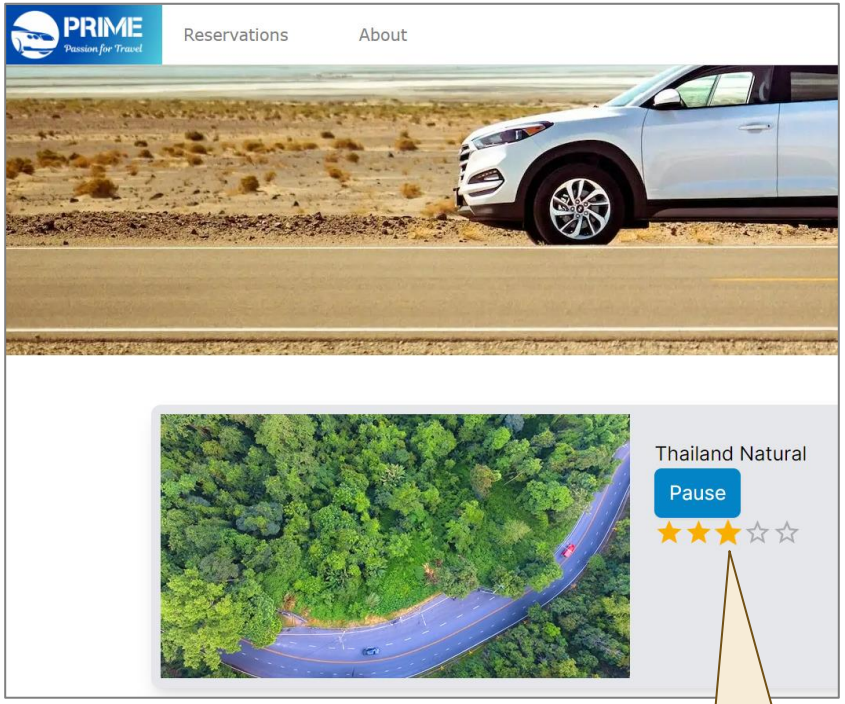
การเปลี่ยน rating ทำให้ effect ทำงานซ้ำ ๆ ด้วย แม้ว่าค่า play/pause จะเท่าเดิมก็ตาม เพราะการเปลี่ยน rating ทำให้ `TravelCard` ถูก re-render และ `VlogPlayer` จะถูก re-render ด้วย



# Dependencies in useEffect

- ใน `components\VlogPlayer.tsx`
  ใส่ dependency array
  - ให้ effect ทำงานเมื่อ
    ค่า `isPlaying` เปลี่ยน

```
useEffect(()=> {
    //alert('width is ' + vdoRef.current?.videoWidth)
    if(isPlaying) {
        alert('Play VDO')
        vdoRef.current?.play()
    }
    else {
        alert('Pause VDO')
        vdoRef.current?.pause()
    }
}, [isPlaying])
```



# Clean-Up Function in useEffect



- ลองใช้งาน Effect ที่ต้องการการ clean up เช่น เมื่อมีการ add Event Listener

```
const [winwidth, setWinwidth] = useState(0)
useEffect(()=>{
    const handleWinWidthChange = ()=>{
        setWinwidth(window.innerWidth);
        alert('window width' + window.innerWidth)
    }

    window.addEventListener("resize", handleWinWidthChange)

    return ()=>{
        window.removeEventListener("resize", handleWinWidthChange)
    }
},[])

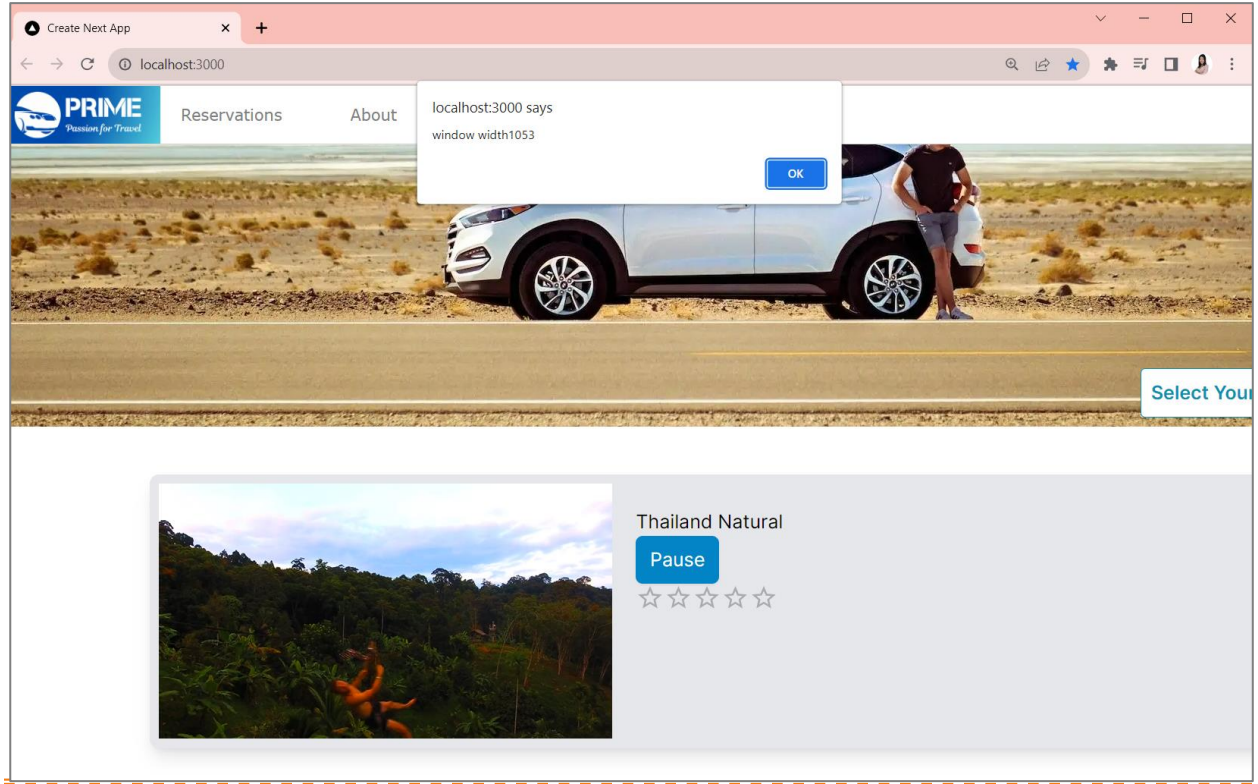
return (
    <video className='w-[40%]' src={vdoSrc} ref={vdoRef} muted
```

Effect return เป็น function ที่ทำการ clean up

Effect ทำงานเมื่อ render ครั้งแรก ตอน mount component

# Clean-Up Function in useEffect

- ทดลองการทำงาน



# Other Built-in Hooks



- React และ Next.js มาพร้อมกับ Built-in Hooks อื่น ๆ เช่น
  - `usePathname` React Hook สำหรับอ่าน URL Path ปัจจุบัน
  - `useCallback` React Hook สำหรับการ cache function definition ข้ามการ re-render
  - `useContext` React Hook สำหรับอ่านและ subscribe เพื่อใช้ค่าใน context ซึ่ง parent component มีให้แก่ component อื่นลงไปใน tree
  - `useMemo` React Hook สำหรับการ cache ผลลัพธ์ของการประมวลผล calculaton ข้ามการ re-render

  ฯลฯ

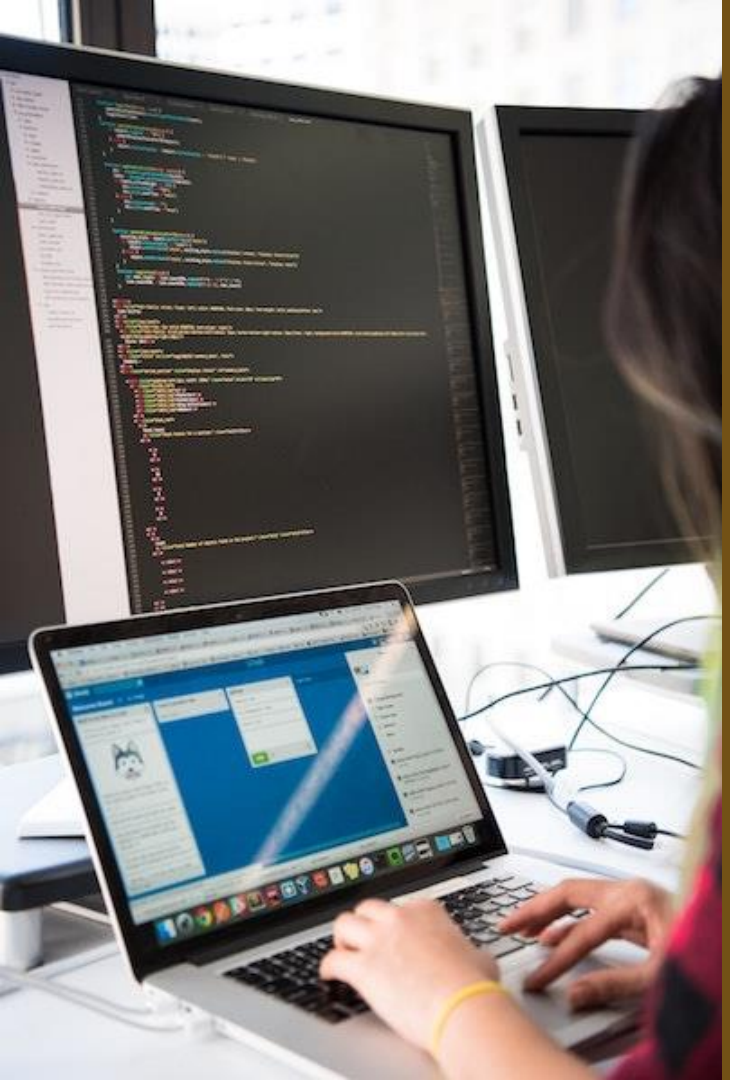
# Custom Hooks



- นอกเหนือจาก Built-in Hooks แล้ว นักพัฒนาอาจต้องการ Hook สำหรับวัตถุประสงค์การทำงานอื่นที่เฉพาะเจาะจงกับ application
  - ดึงข้อมูลสำหรับ application
  - Track การ online ของ user
  - เชื่อมต่อการจัดการ chat room

Custom Hooks สำหรับ re-use logic

- การใช้งาน Hook ที่ให้รูปแบบการทำงานที่ใช้งานบ่อยใน application ทำให้นักพัฒนาต้อง code การทำงานของ Hook แบบเดิม ๆ ซ้ำกันหลาย ๆ รอบ สำหรับใช้ในหลาย component
  - ดึง logic ที่ใช้งาน hook ในรูปแบบที่ common กันในหลาย component ออกมา แยกเขียนเป็น Custom Hooks
  - นำ Custom Hook ไปใช้งานในหลาย component ได้
  - ตั้งชื่อ Custom Hook โดยใช้ React naming conventions ขึ้นต้นด้วย use...

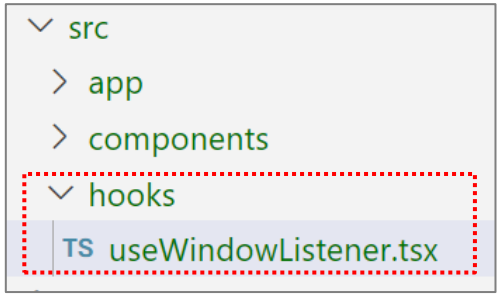


# Workshop 8.4:

# Custom Hooks



- สร้างโฟลเดอร์ hooks และสร้างไฟล์ hooks\useWindowListener.tsx เพื่อเป็น custom hook
  - นำ useEffect ใน components\VlogPlayer.tsx ที่ทำงานกับ window listener เดิมใน มาไว้ใน hooks\useWindowListener.tsx




```tsx
import { useEffect, useState } from "react";
export function useWindowListener() {
    const [winwidth, setWinwidth] = useState(0)
    useEffect(()=>{
        const handleWinWidthChange = ()=>{
            setWinwidth(window.innerWidth);
            alert('window width' + window.innerWidth)
        }

        window.addEventListener("resize", handleWinWidthChange)

        return ()=>{
            window.removeEventListener("resize", handleWinWidthChange)
        }
    },[])
}
```

# Custom Hooks

- ในไฟล์ hooks\VlogPlayer.tsx เรียกใช้ useWindowListener

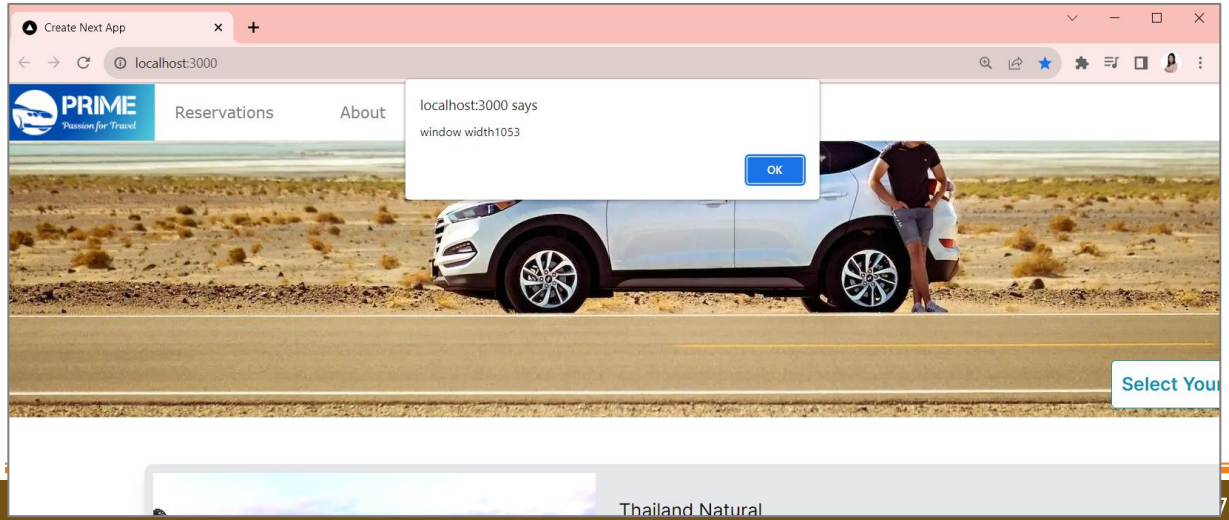
```
'use client'
import { useRef } from 'react'
import { useEffect, useState } from 'react'
import { useWindowListener } from '@/hooks/useWindowListener'
export function VlogPlayer(
    {vdoSrc, isPlaying}: {vdoSrc:string, isPlaying:boolean} ) {

    const vdoRef = useRef<HTMLVideoElement>(null)

    useWindowListener()

    useEffect(()=> {
```

- ทดสอบการทำงาน



# Custom Hooks



- ไฟล์ hooks\useWindowListener.tsx ปรับให้รับ data จาก component ที่เรียก
  - ฟัง event ที่ระบุมา และดำเนินการตามที่กำหนดโดย event listener ที่รับมา

```
import { useEffect, useState } from "react";
export function useWindowListener(eventType:string, listener:EventListener) {
    useEffect(()=>{

      window.addEventListener(eventType, listener)

        return ()=>{
            window.removeEventListener(eventType, listener)
        }
    },[eventType, listener])
}
```

# Custom Hooks

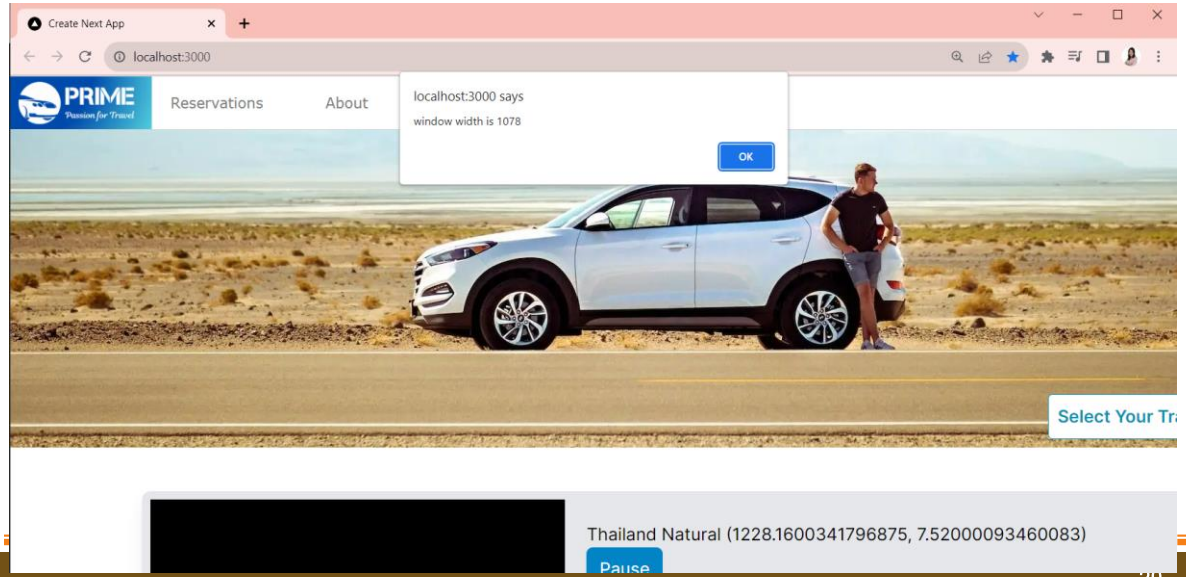
- ในไฟล์ `VlogPlayer.tsx` เรียกใช้ `useWindowListener`
  - ฟัง event การ resize window

```tsx
export function VlogPlayer(
    {vdoSrc, isPlaying}: {vdoSrc:string, isPlaying:boolean} ) {

    const vdoRef = useRef<HTMLVideoElement>(null)

    useWindowListener('resize',
    (e)=>alert('window width is ' + (e.target as Window).innerWidth))
```

- ทดสอบการทำงาน



# Custom Hooks



- ในไฟล์ TravelCard.tsx เรียกใช้ useWindowListener
  - ฟัง event การ move pointer นำค่า x, y มาแสดง

```
import { Rating } from '@mui/material';
import { useWindowListener } from '@/hooks/useWindowListener'

export function TravelCard () {

    const [playing, setPlaying] = useState(true)
    const [rating, setRating] = useState(0)
    const [pointerPosition, setPointerPosition] = useState({ x: 0, y: 0 });

    useWindowListener('pointermove',
    (e)=>{setPointerPosition({ x: (e as PointerEvent).clientX, y: (e as PointerEvent).clientY })})

    return (
        <div className='w-[80%] shadow-lg mx-[10%] my-10 p-2 rounded-lg bg-gray-200
        flex flex-row'>
        <VlogPlayer isPlaying={playing} vdoSrc="/video/ThailandNatural.mp4"></VlogPlayer>
        <div className="m-5">
            Thailand Natural ({pointerPosition.x}, {pointerPosition.y})
            <button className="block rounded-md bg-sky-600 hover:bg-indigo-600 px-3 py-2
```

# Custom Hooks

- ทดสอบการทำงานของ application

